

ML System Design in a Hurry

Embeddings

What embeddings are, how they're trained, and how they show up in search, recommendations, classification, and RAG systems.

Embeddings are one of the most important ideas in modern machine learning, and they show up everywhere in ML system design interviews. Recommendations, search, RAG, classification, fraud detection, ad ranking. Pull on any of them hard enough and you hit an embedding. If you can speak fluently about what they are, how they're trained, and how they're served, you'll handle a big chunk of the interview surface area.

So what actually is one? An **embedding is a list of numbers that represents something** — a user, a product, a word, an image, a search query — in a way that captures its meaning. Instead of describing a user by their ID or a checklist of attributes, you describe them with something like 128 floating-point numbers. The key property is that similar entities get similar numbers. A user who behaves like another user ends up with a nearby vector. And that turns out to be tremendously useful across the board.



This is a high-level description of embeddings, and the devil is in the details. Plenty of candidates understand embeddings purely as a word2vec-style $\text{man} - \text{woman} + \text{queen} = \text{king}$ or a hand-wavy "semantic understanding". This is frequently below the bar for most companies and you'll get caught out on follow-up questions. You need to understand how embeddings are actually created to speak confidently about how they can be used, even if these shorthands are useful in places. We'll cover that here!

Why Embeddings Exist

Before embeddings, the standard way to represent categorical data in ML was one-hot encoding. One-hot encoding is just what it sounds like. If you have 10M users, every user is a 10M-dimensional vector with a single 1 and a bunch of 0s. Now, every dimension or position in that vector means something. The first dimension means user **A**, the second means user **B**, and so on. You can plug this into a linear model and it will learn something about each user. This sounds bad and it is.

First, it's wasteful. Your model has to learn something about every user from scratch, with no shared structure. There's no notion that user **A** and user **B** might be similar just because they've both watched a lot of cooking videos.

Second, it doesn't **generalize** to new entities. A brand-new user has never appeared in your training data, so the model has nothing to say about them. You need a new dimension and a newly trained model. This sucks.

Embeddings fix both problems at once. The model learns a compact representation where similar users cluster together in the vector space. You can compute meaningful distances, average embeddings to get a "this kind of user" vector, and, with the right training setup, embed new users from their attributes at inference time.

To make all of this concrete, here's roughly what a canonical embedding answer sounds like in an interview:

"I'd represent each user and each video as a 64-dimensional embedding. The user tower takes demographics and recent watch history; the video tower takes content features. I'd train them so that the dot product of a user embedding and a video embedding predicts whether the user watched the video."

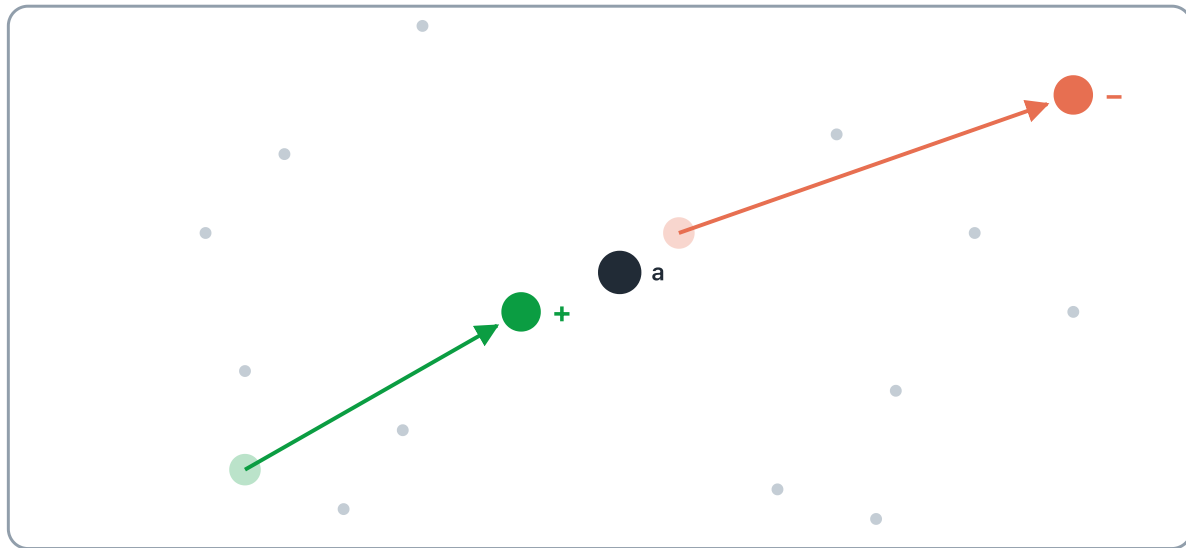
That sentence, or some version of it, will land well in about half the ML system design interviews you'll take. The rest of this doc is about what each piece of that sentence actually means.

How Embeddings Get Trained

The gist of training embeddings looks like this: you pick some notion of what "similar" should mean for your problem, and randomly assign embeddings to each entity in your training set. Then you move those embeddings around until, in aggregate, the similar entities are close together and the dissimilar entities are far apart. How you do this varies, whether you're using a sophisticated graph model or plain old matrix factorization, but the mechanic is exactly the same.

Pull positives in, push negatives out

the embedding space, mid-training



● anchor ● positive (pulled in) ● negative (pushed out) ● other points



Worth pausing on this. "Similar" is whatever *you* decide it is when you set up training, and it does not necessarily mean "semantically similar". Train on co-watch data and similar means "tends to get watched by the same people." Two totally different-looking videos can end up next to each other because they're both late-night comfort viewing. Train on co-purchase data and similar means "bought by the same shopper," which is why toothbrushes and toothpaste end up close. Not because they're alike, but because they show up in the same carts.

Real production embeddings are often trained with multiple objectives stacked together. A watch-completion objective, a like objective, a diversity regularizer, a fairness constraint. After a while it's genuinely hard to say what the embedding is "about" in plain English. But the embedding's structure comes from the objective you used to train it. This is important.

Run this enough times over enough data and something interesting happens. You never told the model to build categories or discover structure, you just gave it pairwise nudges. But because every nudge pulls similar things toward each other and pushes dissimilar things apart, the whole space self-organizes. Things that are similar to each other end up clustered together, and those clusters sit in consistent positions relative to each other. Cats, dogs, and wolves end up near each other; fruits end up in a different neighborhood; verbs of motion in another.

This structure is what makes embeddings useful for retrieval, clustering, recommendations, and the other applications we'll get to later. The approaches below all end up producing it, but they each get there differently.

Matrix Factorization

Matrix factorization is the classical approach for learning embeddings directly from an interaction matrix. The canonical setup is a recommender system. Picture Netflix. You have a roughly $100M \times 10K$ matrix R where rows are users, columns are movies, and a cell is 1 if that user watched that movie. Most cells are empty, because no user has watched more than a tiny slice of the catalog. What you want out of this is a 64-dimensional embedding for every user and every movie.

What we want to do is find two smaller matrices that, when multiplied together, approximate R . If we can do this, we're capturing the essence of the data in a much smaller space. Specifically, a $users \times 64$ matrix U and a $64 \times movies$ matrix V^T such that $U \cdot V^T \approx R$. The rows of U are your user embeddings, the columns of V^T are your movie embeddings. That's matrix factorization.

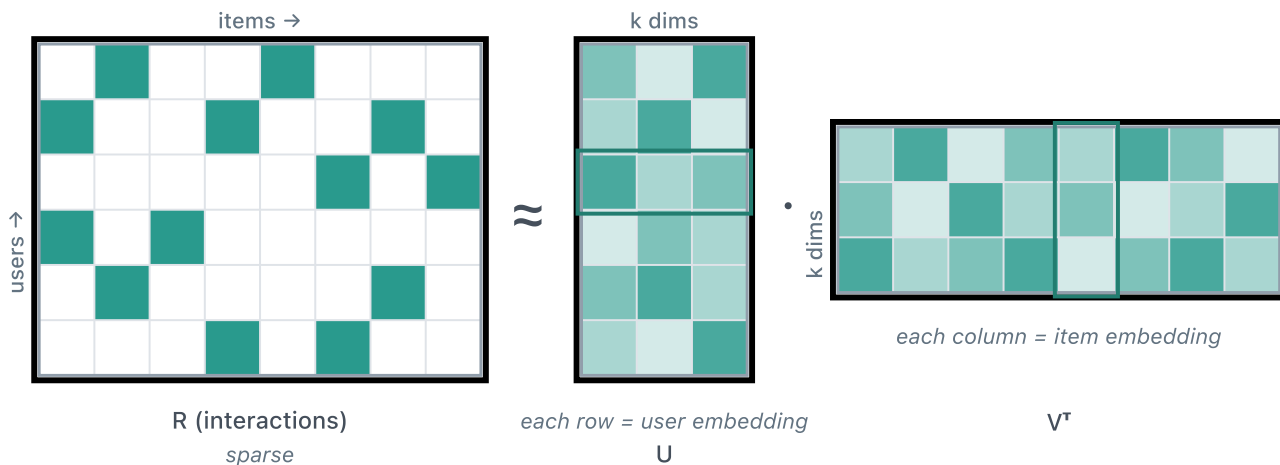
The reason this works is compression. You're forcing 100M users' behavior to be summarized by just 64 numbers each. You cannot give every user a unique fingerprint at that size, so the decomposition has to find shared patterns: "people who watch a lot of stand-up specials," "people who watch true crime late at night," "people who mainline sci-fi." Users whose real behavior in R is similar end up with similar rows in U , because that's the only economical way to approximate their rows of R . Movies watched by similar users end up with similar columns in V^T for the same reason. The pull-similar-together dynamic from the overview isn't implemented via pairwise nudges here. It's baked into the low-rank constraint itself.

The steps for training are:

1. **Build the interaction matrix** R of size $users \times items$. Cells are 1 where a user interacted with an item (click, watch, purchase), unknown elsewhere.
2. **Pick an embedding dimension** k (e.g. 64 or 128).
3. **Initialize** two random matrices: U of shape $users \times k$ and V of shape $items \times k$.
4. **Optimize** U and V to minimize reconstruction error on observed cells: $\|R - U \cdot V^T\|^2$, typically via Alternating Least Squares (ALS) or SGD. Regularization is usually added to

keep weights small.

5. **Read out** the embeddings. Row i of U is user i 's embedding; row j of V is item j 's embedding.



These methods are cheap, well-understood, and still widely deployed. They're a perfectly reasonable baseline answer in interviews for retrieval problems. That said, unless the baseline already solves the problem, there are almost always more powerful representations you can learn with the approaches below, and interviewers will usually expect you to reach for them.



In a recommendation interview, starting with "I'd train user and item embeddings via matrix factorization on the interaction matrix, then use approximate nearest neighbor search for retrieval" is a completely reasonable baseline. You can layer on two-tower and neural approaches after you've established it.

Two-Tower and Contrastive Learning

We can do a lot better. The broader paradigm here is **contrastive learning**. You have pairs (or triplets) that you know should be similar or dissimilar, and you train a model to pull similar things together and push dissimilar things apart in the embedding space. Where co-occurrence gave us the signal implicitly (things in the same window are similar), contrastive learning makes it explicit. Whatever your domain, if you can say "these two things go together and this third thing doesn't," you have enough to train an embedding.

The dominant architecture that actually uses this paradigm in production is the **two-tower** (or dual encoder) model. You build two neural networks, the "towers," one that encodes queries (or users) and one that encodes candidates (or items). The towers can be totally different, with different inputs, different features, even different architectures. A user tower might eat demographics and recent watch history through an MLP. An item tower might eat title text through a transformer and combine that with categorical video metadata.

The critical thing is that both towers output into the *same vector space*. That is the whole point of the architecture, and it's worth slowing down on. Same vector space means both embeddings have the same dimensionality (say, 128 floats) and, more importantly, those dimensions *mean the same thing* in both towers. Dimension 7 in a user embedding corresponds to the same latent concept as dimension 7 in an item embedding, so a dot product between them produces a meaningful score of how well the two match. Having these embedding produced together is what makes them useful.

The training recipe:

1. **Collect positive pairs.** From interaction logs: (query, candidate) where the user engaged with the candidate (clicked, watched, purchased). Or, for self-supervised setups, synthesize positives by augmenting the same input (two crops of an image, two paraphrases of a sentence).
2. **Build the encoders.** Usually two, one per side. When both sides are the same kind of thing (image-image, sentence-sentence), a single shared encoder — a "siamese" setup — is common instead.
3. **Forward pass** a batch through the encoder(s), producing embeddings.
4. **Compute similarities** between every pair in the batch. The diagonal entries are positive pairs; off-diagonal entries are "in-batch negatives," other candidates in the batch that don't belong to this query.
5. **Apply a contrastive loss.** Two dominate. **Triplet Loss** pushes an anchor closer to its positive than to a negative by some margin. InfoNCE treats it as classification: given an anchor and a batch of candidates (one positive, the rest negatives), the model has to pick the positive. InfoNCE is the more common choice for large-batch training because the in-batch negatives give you lots of "free" supervision.
6. **Deploy.** Pre-compute every candidate embedding offline and load them into an ANN index. At serve time, run only the query tower on each incoming request and do a

nearest-neighbor lookup.



Two-tower is everywhere. YouTube recommendations, LinkedIn feed retrieval, semantic search, many RAG systems. The appeal is that you can pre-compute all the candidate embeddings offline, index them for fast nearest-neighbor lookup, and then at serve time only run the query tower and do a vector search. This is what makes sub-100ms retrieval over billions of items possible. The broader contrastive paradigm shows up wherever the training signal is "these two things go together" rather than "this thing has this label." CLIP learns joint image-text embeddings this way. SimCSE learns sentence embeddings. Many face recognition systems learn identity embeddings.

The tricky part is negative sampling. You have plenty of positive pairs (user watched this video), but you need negatives for the model to learn anything. Random sampling is easy but weak.

Hard negatives are items the model currently thinks are relevant but actually aren't, and they're what drive quality. Mining them well is non-trivial.



Finding hard negatives *at training time* is its own headache. To know which candidates are "hard" for the current model, you'd need to run nearest-neighbor search over the full candidate index, but that index is built from the current model's embeddings, which are changing with every gradient step. Most production setups compromise by mining hard negatives periodically (say, every few thousand steps) using a frozen snapshot of the model, mixing them into the training batches until the next refresh, and repeating. In-batch negatives do a lot of the work in the meantime, which is why two-tower works at all before you've perfected the mining pipeline.

Graph Embeddings

Some data comes with explicit structure. Users follow users, papers cite papers, products get bought together, transactions flow between accounts. Graphs are pervasive in computer science because they model real-world data so well.

That structure is real signal beyond whatever pairwise "these two things are similar" setup you'd otherwise cook up, and graph embeddings are how you train embeddings that actually use it. The two families worth knowing differ mostly in how they handle new nodes.

Transductive methods learn one embedding per node in your training graph, and that's it. The classic examples are node2vec and **DeepWalk**, which treat the graph like a corpus. You do biased random walks from each node and feed the resulting sequences into a word2vec-style objective. Nodes that keep showing up in the same walks end up with similar embeddings. These methods are simple, cheap, and work well when your graph is relatively stable. The catch is that if a new user joins or a new paper gets published, you have no embedding for them until you retrain. Repeating here: **you only get embeddings for the nodes that you trained on.**

Inductive methods learn a *function* that produces an embedding from a node's features and its local neighborhood, rather than a lookup table. GraphSAGE and most modern Graph Neural Networks (GNNs) sit here. At inference time, you embed a new node by looking at its

features and the embeddings of its neighbors, no retraining required. This is usually the right answer in production settings where the graph is constantly growing.

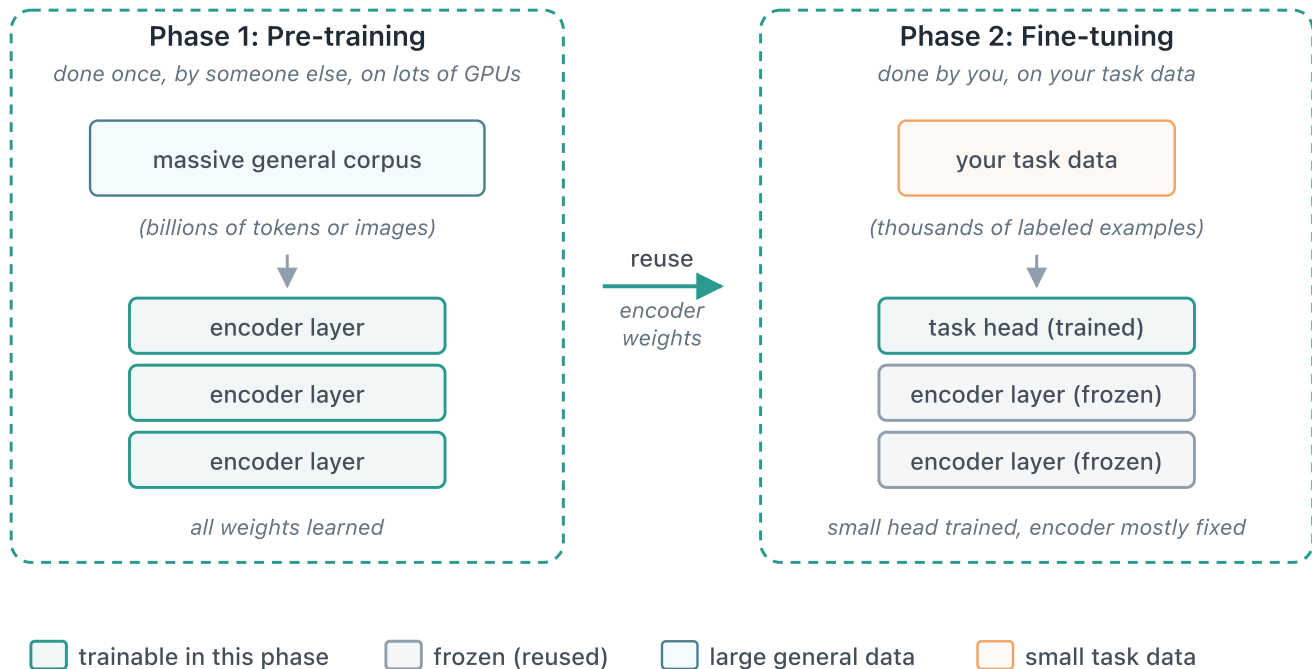
In an interview, flagging the transductive vs inductive distinction is what moves you from "I'd use node2vec" to "I'd use a GNN like GraphSAGE because it's inductive and we can embed new users without retraining." Cold start comes for free with inductive methods. With transductive ones you either retrain often or fall back on a content-based embedding for new nodes until the next refresh. Graph embeddings also compose nicely with the other approaches on this page. A common production pattern is to use a GNN to produce a node embedding, then use that embedding as a feature in a two-tower or ranker downstream.

Pre-trained + Fine-tuning

For a lot of applications, you don't actually need to train embeddings fully from scratch. Say you're building a social network and you want an embedding for every post so you can do semantic search, recommend similar posts, or cluster them for topic discovery. A post is mostly text, and the text is mostly English, and the English has the same grammar and vocabulary it has everywhere else on the internet. A model trained on a billion pages of that same language already knows most of what your embedding needs to know. You're not trying to re-teach it that "startup" and "founder" are related concepts. You just need it to pick up the ways your specific users write about them.

That's the bet behind pre-training plus fine-tuning, and it's where a huge amount of modern ML actually starts. Someone else spent millions of GPU hours training a general-purpose encoder on the open web, and you get to use those weights as a starting point. BERT gives you contextual word embeddings. CLIP gives you image and text embeddings in a shared space. sentence-transformers give you sentence-level embeddings that work surprisingly well out of the box. For most domains, you should not be training embeddings from scratch in the modern era.

The typical flow looks like this. You start from a pre-trained encoder, fine-tune it on your task-specific data (often using a contrastive or two-tower setup), then deploy. This gets you 80% of the way with 10% of the compute and data.



"For the text side I'd start from a pre-trained sentence-transformer. It already knows English semantics from hundreds of millions of examples. Then I'd fine-tune it on our query-document click data with an InfoNCE loss to adapt it to our domain."



This is also where embeddings from LLMs come in. OpenAI, Cohere, and Hugging Face all sell embedding endpoints. They're great for prototypes and smaller-scale systems, but at large scale you'll usually want to host your own model, both for cost and for the ability to fine-tune on your data.

In Practice

Now that we've covered how embeddings are trained, let's look at how they actually get used. Almost every use case you'll see in a system design interview reduces to one of three patterns.

The first is **embeddings as features**. You feed the embedding into a downstream model as a compact, semantically-rich input, and it replaces the one-hot-plus-handcrafted-features approach that used to be standard. This is really useful for transferring knowledge from one task to another. As an example: if you have embeddings trained on co-purchase data (stuff people bought together), the embeddings may be useful for detecting substitutes. People

rarely are buying two products that are directly substitutes for each other, so the task is informative.



In large companies, embeddings are often used to "share" between teams and systems. A rich embedding might be trained for recommendations, and it's easy for a nearby team like content moderation to try using it in their models. Because of this, it's common in interviews for you to just assume certain embeddings exist: some team likely needed the data and has trained a decent embedding which you can use as a feature in your model.

The second is **embeddings for clustering**. You group or segment entities by proximity in the embedding space, which is useful any time you want to ask "which of these things are similar to each other?" User segmentation groups similar users. Topic discovery groups similar documents. Near-duplicate detection finds pairs that sit within some threshold of each other. You rarely need a fancy clustering algorithm once you have good embeddings. Plain k-means usually does the job.

The third is **embeddings for retrieval**, and it's the big one. You embed your candidates offline, index them in a vector store, embed the query online, and do approximate nearest neighbor search. This is the retrieval stage of every modern recommendation system, most search systems, and every RAG pipeline. Two variants are worth naming.

1. Semantic search is retrieval applied to search queries. Classic keyword search doesn't understand meaning, but embeddings do, and most production search systems use a hybrid. Combining the semantic understanding of the embeddings with the lexical understanding of the keywords is a powerful way to get a good search experience.
2. RAG is retrieval applied to LLM context. You embed your documents, embed the user's question, retrieve the top-k nearest documents, stuff them into the prompt. The retrieval quality of a RAG system is almost entirely a function of how good your embeddings are. If your embeddings don't know that "401k withdrawal" is related to "retirement account distribution," no prompt engineering will save the LLM.

Dimensionality

Embedding dimension is a real tradeoff, and one that will definitely come up in practice and occasionally in interviews.

Higher dimensions can represent more nuance but cost more to train, store, and search. Lower dimensions are cheaper but can lose important structure. In practice, most production embeddings sit in the 64-1024 range, with 128-512 being the sweet spot for most applications.

- **Small (64-128):** Good for high-QPS retrieval where serving cost dominates. Typical for huge-scale recommendation retrieval.
- **Medium (256-512):** Default for most tasks. Good balance of quality and cost.
- **Large (1024+):** LLM-style embeddings, CLIP, high-stakes ranking where quality matters more than cost.

There's also a more recent technique called Matryoshka embeddings, where a single embedding is trained so that its first 64 dimensions, its first 128, its first 256, etc. are all valid embeddings on their own. This lets you use a small prefix for fast coarse retrieval and the full embedding for re-ranking. Worth mentioning if you're talking about a system where latency and quality both matter.

Evaluation

Evaluating embeddings directly is hard because the embedding space itself doesn't have a ground-truth structure beyond the loss function they were trained under, which isn't really helpful. In the vast majority of cases you'll measure the embedding by how effective it is toward the task you care about.

Intrinsic evaluation measures properties of the embedding space itself. For word embeddings, you can look at analogies (king - man + woman $\approx$ queen) or word similarity datasets. For sentence embeddings, benchmarks like MTEB give you a standard suite. These are useful for quick iteration during training, but they don't always correlate with production performance.

Extrinsic evaluation measures performance on the downstream task you actually care about. For a retrieval-trained embedding, that means recall@k on held-out query-document pairs. For a classification embedding, it's the accuracy of the downstream classifier. This is what really matters, and what you should anchor your evaluation on in interviews.



In interviews, be cautious of proposing an academic research exercise of trying to assess your embeddings in the abstract. You'll usually go with extrinsic evaluation (recall@k,

downstream task accuracy, online A/B metrics) and layer in a few simple intrinsic sanity checks — nearest-neighbor spot checks for known queries, cosine-similarity spread on a held-out set — to confirm the embeddings are reasonably spread and clustering the things you'd expect. Tie the headline number back to the end-to-end metric you actually care about. See the [Evaluation core concept](#) for more on this.

Serving Embeddings

Once you have good embeddings, you still have to get the right embedding to the right query within a tight latency budget, keep them fresh as the world changes, and handle entities that didn't exist at training time. Each of those is its own problem.

Offline indexing. Most candidate-side embeddings (items, documents, "users-you-might-know" candidates) are pre-computed in a batch job and loaded into an approximate nearest neighbor index. At query time you embed the query, ask the index for the top-k nearest vectors, and return those candidates. The [Vector Databases deep dive](#) covers the mechanics (HNSW graphs, IVF partitioning, quantization, recall-latency tradeoffs) and is worth a read if an interviewer pushes on serving.

Refresh. Embeddings go stale. Item embeddings need to be refreshed when the item changes (new title, new metadata) or when the underlying embedding model is retrained. User embeddings drift as user behavior evolves. In practice, you'll often run a scheduled batch job to re-embed everything periodically, plus an event-driven path for hot updates. This is the part most candidates skip in interviews, and it's often where senior interviewers dig in.

Online embedding updates. In domains where freshness is critical (short-form video, news, fraud) even nightly batch refresh is too slow. The aggressive pattern is to serve embeddings out of a parameter server and update them online as user interactions arrive. The encoder layers stay mostly frozen, but the embedding tables themselves get updated in near real-time. TikTok's [Monolith](#) system is the canonical example, and it's a big part of how TikTok's recommendations adapt so fast. This is a lot of infrastructure to build, but it's the right answer when an interviewer pushes specifically on "how do you adapt to behavior that changes within minutes?"

And lastly, we're often concerned with **Cold start**. Brand-new users or items have no interaction history, so behaviorally-trained embeddings don't help them. The usual fix is a

content-based embedding (from attributes or raw content) that you can compute without any interaction data, then blend in the behavioral embedding as interactions accumulate.



The cold-start story for embeddings is a classic interview probe. "How does this work for a new video with 0 views?" There's usually a continuum from "what's the weakest embedding we can start with" to "how can we incorporate all that we've learned about this video so far (without retraining our embeddings)".

Summary

Embeddings are learned dense vectors that let you represent entities in a way that captures semantic or behavioral similarity. They've become the default representation layer in modern ML because they solve the generalization and sparsity problems of one-hot encodings while enabling fast similarity search.

You'll see them trained via co-occurrence, two-tower, and contrastive methods, often starting from pre-trained models and fine-tuned on your specific task. They power retrieval in recommendation and search, classification via shared representations, and the retrieval step in every RAG system.

In interviews, be specific. Name the loss function. Explain your negative sampling strategy. Pick a dimensionality with a reason. Have a story for how embeddings get refreshed and how cold-start cases are handled. The goal is to show the interviewer that you've actually trained and served embeddings in production, not just read about them.

Embeddings are also a gateway concept. Once you're comfortable with them, problems like **Video Recommendations** and harmful content classification start to look a lot more tractable.